

SOFTWARE ENGINEERING FOR MACHINE LEARNING · COMPANION READER

Agentic AI, SAGA & Blackboard

How AI agents reason, act, and coordinate across distributed systems

Session 7 · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. The slides give you the diagrams; this reader gives you the *why*. Every slide concept is expanded into plain English. Every slide example is worked out step by step. Five colour-coded boxes guide you through each concept:

- **Blue** — “intuition” — states the core idea in one breath.
- **Amber** — “everyday picture” — ties the idea to real life before any diagram.
- **Green** — “worked example” — shows a concrete scenario with every step shown.
- **Red** — “watch out” — flags the common mistake.
- **Purple** — “key takeaway” — the one sentence to remember.

Read alongside the slides, in order.

1 What this session covers

This session covers three big ideas in modern AI systems. First, we look at **Agentic AI**: what it is and how it differs from a simple chatbot. Second, we study the **SAGA pattern**: a technique from distributed databases that now powers multi-agent AI workflows. Third, we learn the **Blackboard pattern**: a way for specialist agents to share knowledge and solve hard problems together.

By the end you should be able to answer four questions. What makes an AI system “agentic”? How does an orchestrator break a task into steps? What happens when one step fails? How does a blackboard let independent agents collaborate?

2 Agentic AI: from chatbots to doers

A chatbot answers a question and stops. You ask it something; it replies; the conversation ends there. An **agentic AI** system is different. You give it a *goal*, and it plans, acts, checks its work, and keeps going until the goal is met.

Three definitions from the slides each capture a piece of this idea. Nvidia says agentic AI “uses sophisticated reasoning and iterative planning to autonomously solve complex, multi-step problems.” OpenAI says these systems “can pursue complex goals with limited direct supervision.” A third definition adds: the system can plan, reflect, and interact with tools or an environment. All three agree: an agent is not just a generator; it is a *doer*.

The intuition

A chatbot is a smart look-up: you ask, it answers. An agentic AI is more like a new employee. Give it a project and it figures out the steps, uses the tools, checks the results, and delivers the work. The key difference is **autonomous multi-step action**.

★ Everyday picture

Think of ordering a meal at a restaurant. A chatbot is like a waiter who reads the menu and takes your order—one exchange, done. An agentic AI is like a personal chef. You say “plan dinner for eight with one vegetarian guest.” The chef plans the courses, shops, cooks, and adjusts if something burns. The chef acts, reacts, and keeps the goal in view the whole time.

2.1 Chatbots vs. Agentic AI: side by side

Feature	Chatbot	Agentic AI
Goal type	One exchange, one answer	A complex, multi-step goal
Planning	None	Breaks the goal into sub-tasks
Tool use	Rare or none	Tools, APIs, databases, code
Memory	Short context window only	Can store and retrieve state
Self-correction	None	Checks output; revises if wrong
Supervision	Every turn	Minimal; runs autonomously

2.2 Generative AI vs. Agentic AI

Generative AI takes a prompt and produces an output: “Tell me what to do; I will generate an answer.” **Agentic AI** adds three things on top of that foundation: *reasoning*, *tools*, and *feedback loops*.

The slide’s formula:

$$\text{Agentic AI} = \text{Generative AI} + \text{Reasoning} + \text{Tools} + \text{Feedback Loops}$$

Here, **Reasoning** means the system can think through a plan step by step before acting. **Tools** are external things the agent can call: calculators, search APIs, databases, and code runners. **Feedback Loops** mean the agent checks its own output and tries again if something is wrong.

2.3 Why LLMs power modern agents

A **Large Language Model (LLM)** is a model trained on large amounts of text. It turns out LLMs have exactly the skills an agent needs. They can reason step by step before acting. They can follow a natural-language goal and turn it into actions. They can handle new tasks without retraining. They can format output as a **tool call**: a JSON object that tells the system to run an external function. They can evaluate their own output and revise it.

2.4 Memory and state

An agent often needs to remember earlier steps. **Memory** in an agentic system takes several forms. *In-context memory* is the text history inside the prompt window: cheap but limited by the window size. *External memory* (for example, a vector database) lets the agent look up facts from earlier steps or past runs. *Tool-based memory* means the agent calls a “save this” tool that writes to a log.

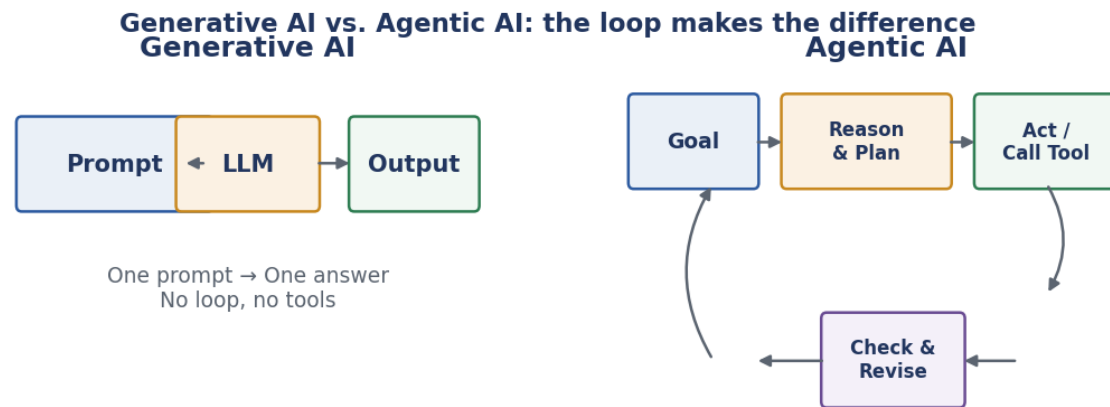


Figure 1: Generative AI vs. Agentic AI. Generative AI produces one output per prompt. Agentic AI wraps that core with a reasoning loop, tool calls, and self-correction—turning a text generator into an autonomous problem-solver.

2.5 Tool use and action execution

Agents call tools by producing a structured JSON output. The system reads it, runs the real function, and feeds the result back.

The general format from the slides:

```
{
  "tool_name": "function_name",
  "arguments": { "param1": "value1" }
}
```

A concrete example: asking for the weather in Bangalore.

Tool call (agent output)	Tool result (fed back in)
<code>{"name": "get_weather", "arguments": {"city": "Bangalore"}}</code>	<code>{"tool_result": "Temperature is 28°C"}</code>

Worked example: weather tool call trace

Suppose an agent needs to check whether it is safe to fly a drone.

Step 1. Agent decides it needs the weather. It outputs the JSON: `{"name": "get_weather", "arguments": {"city": "Bangalore"}}`.

Step 2. System executes the tool. It calls the weather API and gets back: “28°C, clear skies.”

Step 3. Result injected into the prompt. The agent now sees: “Tool result: 28°C, clear skies.”

Step 4. Agent reasons over the result. Output: “Conditions are safe. Proceeding with drone flight plan.”

Step 5. Sense-check. Every step is visible: the call, the result, the reasoning. Nothing

is hidden from the log.

The agent never “knows” the weather directly. It uses a tool and reads the answer, just like a person using a weather app.

⚠ Watch out

Tool results are fed back into the context window. Long tool outputs can fill the context and push out earlier steps. Design your tools to return only what the agent needs, not a full data dump. Also: an agent can hallucinate a tool call for a tool that does not exist. Always validate the tool name before executing it.

Where this shows up in ML. Every modern LLM-based agent framework is built on this exact loop. LangChain, LlamaIndex, OpenAI Assistants API, and AWS Bedrock Agents all work the same way. They generate a tool call as text, parse it, run the function, and feed the result back. The agent’s intelligence is in the text it generates; the tools do the real work.

✓ Key takeaway

Agentic AI = a generative model + a loop: plan, call tools, check results, revise. The loop is the key; without it, the model is just a text generator.

3 The SAGA pattern: distributed transactions made reliable

Before we apply SAGA to AI, we need the original problem it solved.

Suppose you run an e-commerce site. When a customer orders, four things must happen: the order is recorded, inventory is reserved, payment is charged, and shipping is scheduled. Each action lives in a different service with its own database. This is a **distributed transaction**: a set of steps that must all succeed or all roll back, across multiple services.

The naive fix is a global lock: freeze everything while all four steps run. This works but destroys performance at scale. The SAGA pattern is the better answer.

👉 The intuition

A **saga** is a sequence of small local transactions, one per service. Each step commits to its own database right away. If a later step fails, the saga does not roll back by locking everything. Instead, it runs **compensating transactions**: new actions that undo the earlier commits one by one. “Undo by doing the opposite action”, not “undo by erasing.”

★ Everyday picture

Imagine booking a holiday: you confirm a flight, then a hotel, then a rental car. Each booking is paid right away. Now the car company is full. You cannot “un-confirm” the flight by magic—you cancel it manually. That cancellation is the compensating transaction: a real action that reverses the earlier real action. SAGA works exactly this way.

3.1 Why compensating transactions, not rollback?

In a traditional single-database transaction, a *rollback* erases changes as if they never happened. In a saga, each step already committed to a live database. Other services may have already read

and acted on those committed values. Pretending the commit never happened is not safe. So instead you run a compensating transaction: a new action whose effect cancels the old one.

Example: if step 2 reserved 10 items, the compensating transaction for step 2 is “un-reserve 10 items”—a real write, not an erasure.

Worked example: e-commerce order saga with one failure

An order involves four services: Order, Inventory, Payment, Shipping. Payment fails at step 3.

Step 1. Order service commits. Creates the order record. Status: *Created*. Committed to Order DB.

Step 2. Inventory service commits. Reserves the items. Status: *Reserved*. Committed to Inventory DB.

Step 3. Payment service fails. Attempts to charge the card. Card declined. **FAILURE** here.

Step 4. Compensate step 2. Inventory service un-reserves the items. Writes: *Released*.

Step 5. Compensate step 1. Order service marks the order *Cancelled*.

Step 6. Final state. No money charged. No items reserved. No ghost orders. System is consistent again.

Compensation runs in reverse order: undo step 2 before step 1, because step 2 depended on step 1.

SAGA pattern: happy path and compensating rollback

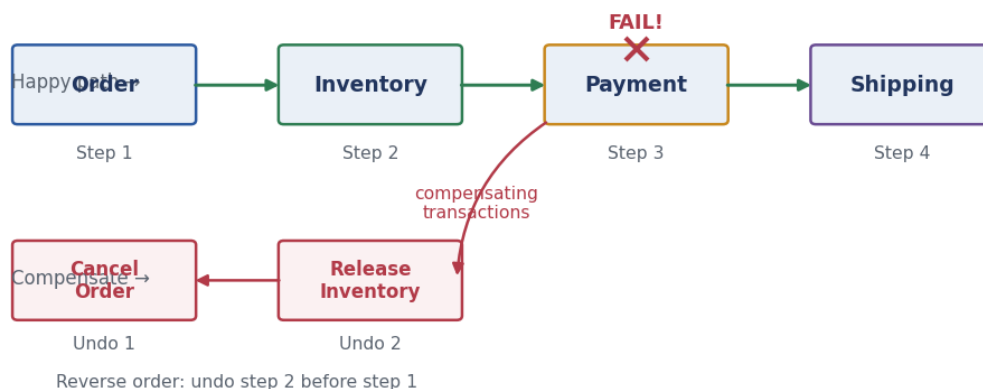


Figure 2: A four-step saga and its compensating path. The happy path (top) commits each local transaction in order. When step 3 fails, compensating transactions run in reverse (bottom) to restore consistency.

3.2 Two ways to implement a saga

3.2.1 Orchestration

In **orchestration**, a central *orchestrator* knows the whole plan. It tells each service what to do, one step at a time. It waits for the result, then sends the next command. If a step fails, the orchestrator sends the compensating commands in reverse.

Think of it as a project manager who holds the full plan and delegates tasks one at a time.

3.2.2 Choreography

In **choreography**, there is no central planner. Each service listens for events and acts when the right event arrives. When service A finishes, it publishes an event; service B is subscribed and starts its step. If something fails, the failing service publishes a failure event and others run their compensating logic.

Think of it as a relay race: each runner starts when the baton arrives, with no coach directing each handoff.

Feature	Orchestration	Choreography
Central coordinator	Yes—one orchestrator object	No—events drive each service
Coupling	Services coupled to orchestrator	Services coupled only via events
Visibility	Easy—orchestrator sees all	Harder—trace events to understand flow
Failure handling	Orchestrator sends compensations	Each service triggers its own undo
Best for	Complex flows with many conditions	Simple, linear pipelines

Where this shows up in ML. SAGA is now a first-class pattern in agentic AI. AWS Prescriptive Guidance explicitly names “SAGA Orchestration” and “SAGA Choreography” (also called prompt chaining) as patterns for multi-agent coordination. The agent framework handles the saga logic; each individual agent is just a service with its own local state and tools.

✓ Key takeaway

A saga splits a distributed transaction into small local steps, each committed immediately. If any step fails, compensating transactions undo the earlier steps in reverse. Orchestration uses a central planner; choreography uses events.

4 SAGA orchestration in agentic AI

Now we connect SAGA directly to multi-agent AI.

Here is the coordination problem from the slides. How do we handle a multi-step task run by multiple AI agents? Each agent has its own local state, tools, and memory. This is the *agentic AI coordination problem*, and it maps exactly onto the distributed transaction problem.

The **SAGA Orchestration pattern for AI** works in five steps:

1. A central **orchestrator agent** receives the overall task.
2. It decomposes the task into sub-tasks.
3. It delegates each sub-task to a **worker agent**—a specialist in research, coding, review, or summarization.
4. Each worker commits its result to shared memory and reports back.

5. If a worker fails, the orchestrator triggers a compensating action: retry, reassign, or roll back.

Worked example: research pipeline with three specialist agents

Goal: “Write a technical report on transformer efficiency.”

Step 1. Orchestrator delegates to Research Agent. Task: “Find five recent papers on transformer efficiency.” Research Agent uses a search tool, writes a summary to shared memory. Status: *Done*.

Step 2. Orchestrator delegates to Coding Agent. Task: “Run the benchmark from the top paper.” Coding Agent reads the summary, runs code, writes benchmark results. Status: *Done*.

Step 3. Orchestrator delegates to Writer Agent. Task: “Draft Section 2 using the research summary and results.” Writer Agent reads both artifacts and produces the draft. Status: *Done*.

Step 4. Orchestrator checks quality. Reads the draft, decides it is good, marks the task complete.

Step 5. What if step 2 had failed? The orchestrator sends a compensating task: “Try the simpler benchmark instead.” This new action is the compensating transaction for step 2.

Each agent is a “service with local state.” The orchestrator is the saga coordinator.

SAGA Orchestration: one orchestrator, many specialist workers

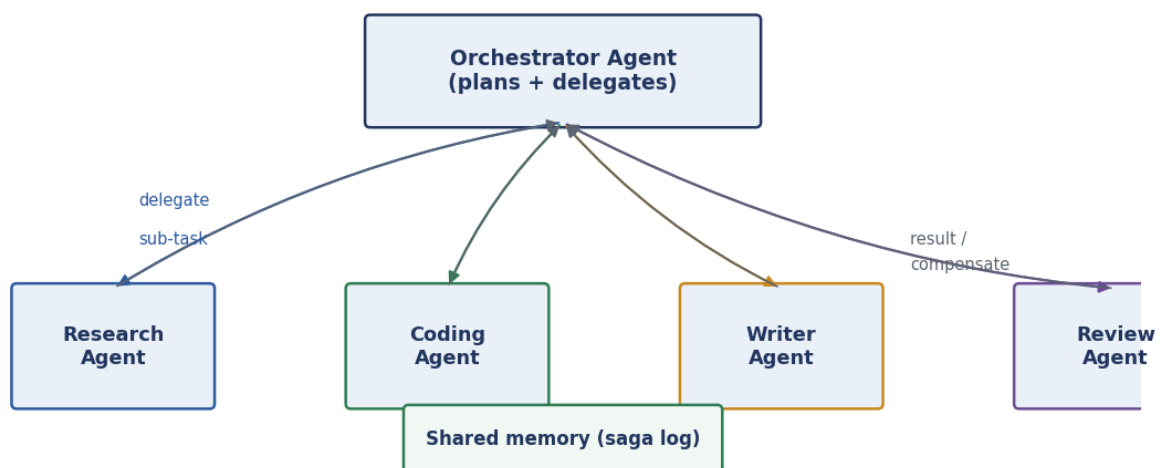


Figure 3: SAGA Orchestration in agentic AI. A central orchestrator decomposes a goal, delegates to specialist workers, and handles failures via compensating tasks. Each worker commits its result independently.

Where this shows up in ML. This pattern is directly used in AWS Bedrock multi-agent systems, CrewAI, and AutoGen. The orchestrator is typically an LLM that decides the plan; the workers are LLMs (or tools) with specific roles. The SAGA logic—compensate on failure, retry, re-delegate—is what makes multi-agent systems reliable in production.

⚠ Watch out

If the orchestrator itself crashes mid-saga, worker agents may have committed partial results. Production systems need a persistent saga log so a recovery process can resume or compensate from the last known good state. Never run a multi-agent saga with no state log.

✓ Key takeaway

In agentic AI, SAGA Orchestration = one orchestrator LLM + multiple worker agents, each a local transaction. Failures trigger compensating tasks sent by the orchestrator.

5 SAGA choreography in agentic AI: prompt chaining

The choreography variant of SAGA maps onto **prompt chaining**. In prompt chaining, each LLM call produces an output that feeds the next call as input.

The slide calls this “Agent Choreography” and says it “resembles the saga pattern in both structure and purpose.” The structure is the same: sequential steps, each building on the last. The purpose is the same: maintain consistency and allow rollback when something goes wrong. But instead of microservices publishing events, we have LLM calls producing text that drives the next call.

The four-step choreography flow from the slides:

1. LLM interprets a complex query and generates a hypothesis.
2. LLM elaborates a plan to solve the task.
3. LLM executes a sub-task (a tool call or a retrieval step).
4. LLM refines the output, or revisits an earlier step if the result is bad.

The compensating mechanism has three options when a result is wrong. The system can retry the step with a different approach. It can revert to a previous prompt and replan. Or it can use an evaluator loop to detect and fix the failure.

📝 Worked example: three-step prompt chain for data analysis

Goal: “Analyse the sales data and explain the main trend.”

Step 1. Step A — Hypothesis. Prompt: “You are given a sales CSV. Generate a hypothesis about the main trend.” Output: “Hypothesis: sales dropped 15% in Q3, likely due to seasonal effects.”

Step 2. Step B — Analysis. Prompt: “Using the hypothesis, write a SQL query to verify the Q3 drop.” Output: a SQL query. Tool runs the query; result: Q3 revenue \$820k, Q2 revenue \$970k. Actual drop: $(970 - 820)/970 \approx 15.5\%$.

Step 3. Step C — Report. Prompt: “The query confirmed a 15.5% Q3 drop. Write one paragraph for a non-technical manager.” Output: plain-English paragraph.

Step 4. Compensating step (if Step B returned a SQL error). System detects the error. New prompt: “The previous SQL had a syntax error. Rewrite it using only standard SQL.” This is the compensating action—a retry prompt

that corrects the failure.

Each step commits its output to the shared context before the next step reads it. The compensation is a new prompt that fixes the bad output from an earlier step.

The intuition

Prompt chaining is saga choreography at the LLM level. Each LLM call is a local transaction. The output is the “commit.” A compensating transaction is a new prompt that says: “go back and fix step N.”

Where this shows up in ML. Prompt chaining is the backbone of multi-step LLM pipelines in LangChain, LlamaIndex, and the OpenAI Assistants API. The evaluator-optimizer pattern adds a judge LLM that checks each step’s output and triggers the compensating retry when needed.

Watch out

Prompt chains can drift: a small error in step 2 gets amplified by step 3. Always validate intermediate outputs before passing them forward. Use an explicit quality check (a short “is this output correct?” prompt) rather than hoping later steps will catch errors.

Prompt chaining = SAGA choreography at the LLM level

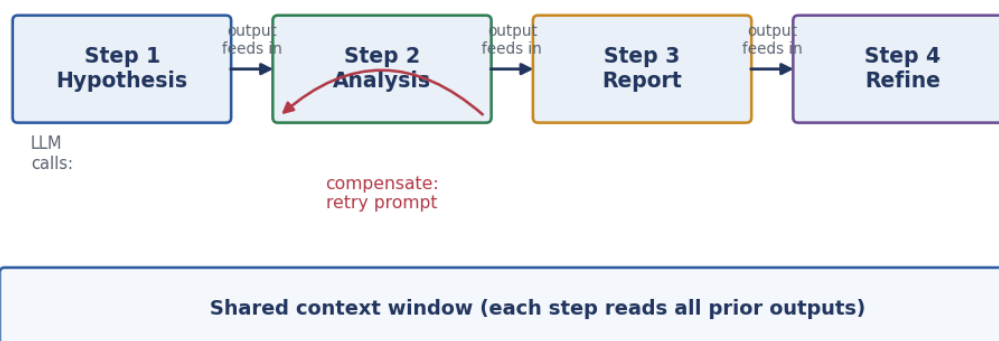


Figure 4: Prompt chaining as SAGA choreography. Each LLM call is a local transaction whose output feeds the next step. The shared context window is the event bus. A failed step triggers a compensating retry prompt (red arrow).

Key takeaway

SAGA Choreography in AI = prompt chaining. Each LLM call is a local transaction; compensating actions are retry prompts. There is no central coordinator—each step drives the next via shared context.

6 The Blackboard pattern: solving hard problems together

Some problems are too complex for any one specialist to solve alone. Speech recognition needs phonetic analysis, language grammar, acoustic modelling, and context understanding—all at once. The **Blackboard pattern** lets many specialist agents work on the same problem without a central planner assigning every step.

The name comes from a vivid mental picture. Imagine a team of experts around a physical blackboard. A hard problem is written on it. Each expert watches the board. When an expert sees something they can contribute, they walk up and add their partial solution. Others react to that new piece of information. This repeats until the full solution emerges.

The intuition

The blackboard is a shared whiteboard that every specialist can read from and write to. No one tells you when to act. You watch the board and step up when you see something you can solve. The solution builds itself from many small contributions.

★ Everyday picture

Think of a hospital emergency room. A patient comes in with many symptoms. The nurse writes the vital signs on a shared chart. The cardiologist reads the chart and adds a note about the heart rate. The radiologist adds the X-ray result. The attending physician reads all the notes and makes a decision. No one “directed” the radiologist to check the lungs—they noticed a relevant symptom and acted. The chart (the blackboard) coordinates everything without a central manager.

6.1 The three core components

Blackboard The central shared memory. It holds the current problem state: partial solutions, hypotheses, intermediate results. All agents read from it and write to it.

Knowledge Sources

Independent specialist agents. Each has a specific skill, set of tools, or model. They watch the board for updates that match their domain. When relevant data appears, they compute and write back.

Controller / Scheduler

A component that watches the board’s state. It decides which knowledge source should act next based on what has just been written. It does not tell agents *how* to solve things—only *when* to act.

Worked example: multi-agent document analysis on a blackboard

Goal: extract the key claims from a scientific paper and check them for consistency. The blackboard holds the current partial analysis.

Step 1. Initial state. Blackboard contains: `raw_text = [full paper text]`.

Step 2. Section Parser (KS-1) acts. It reads the raw text. It writes: `sections = {abstract, methods, results, discussion}`.

Step 3. Claim Extractor (KS-2) acts. It reads the abstract and results sections. It writes: `claims = ["accuracy is 94%", "latency under 10ms"]`.

Blackboard pattern: shared memory + specialist agents + controller

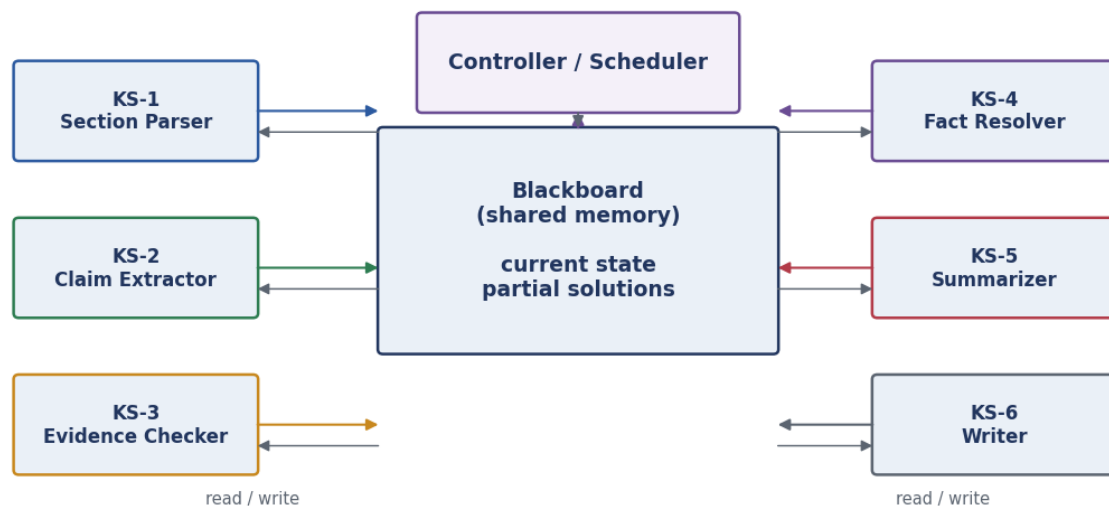


Figure 5: The Blackboard architecture. Specialist Knowledge Sources monitor a shared Blackboard and contribute partial solutions. A Controller decides which agent acts next based on the current board state.

Step 4. Evidence Checker (KS-3) acts. It reads the claims and the methods section. It writes: `evidence_check = ["94% confirmed in Table 2", "latency not reported"]`.

Step 5. Consistency Resolver (KS-4) acts. It reads the evidence check. It writes: `verdict = "Claim 1 supported; Claim 2 unverifiable."`

Step 6. Controller decides at each step. After KS-1 writes sections, the controller sees KS-2 can now act and activates it. After KS-2 writes claims, the controller activates KS-3. The controller schedules; the agents solve.

The final verdict emerged from the blackboard without any agent knowing the full plan upfront.

⚠ Watch out

The blackboard can become a bottleneck. If many agents try to write at once, you need a locking or versioning strategy. Also, a bad scheduler decision can activate the wrong agent too early, producing poor partial solutions that mislead later agents. Design the Controller carefully.

Where this shows up in ML. The Blackboard pattern is used in multi-agent LLM systems where sub-tasks are not strictly sequential. A research assistant system might have a Search Agent, a Summarizer, a Fact-checker, and a Writer. They all share a knowledge base—that is the blackboard. Each reads the current state and contributes when it can add value—no central orchestrator needed. AWS Bedrock shared memory patterns and LangGraph’s shared state graph are modern implementations of this idea.

✓ Key takeaway

Blackboard pattern = a shared memory board + specialist agents who act when they see relevant data + a scheduler who decides who goes next. No agent needs to know the full plan; the solution emerges from many small contributions.

Glossary & symbols

Key terms.

Agentic AI	An AI system that pursues a complex goal by planning, calling tools, checking results, and revising—with minimal human direction at each step.
Agent	One autonomous unit in an agentic system; has a role, tools, and local memory.
Blackboard	The shared memory repository in the Blackboard pattern; all agents read from it and write to it.
Choreography	A saga implementation where each service publishes events and others react; no central coordinator.
Compensating transaction	A new action that reverses the effect of an earlier committed action; the SAGA rollback mechanism.
Controller / Scheduler	The Blackboard component that decides which Knowledge Source acts next.
Distributed transaction	A set of operations across multiple services that must all succeed or all roll back.
Knowledge Source	A specialist agent in the Blackboard pattern; monitors the board and acts on relevant data.
LLM	Large Language Model; a foundation model trained on large text corpora that can reason, follow instructions, and call tools.
Orchestration	A saga implementation where a central orchestrator directs each participant and handles failures.
Prompt chaining	A sequence of LLM calls where each output feeds the next; the AI analogue of saga choreography.
SAGA	A distributed-transaction pattern: a sequence of local transactions each committed immediately, with compensating transactions on failure.
Tool call	A JSON-formatted LLM output that tells the system to run an external function and return the result.

Abbreviations at a glance.

Abbreviation	Meaning
LLM	Large Language Model
SAGA	Sequence of local transactions with compensating rollback
KS	Knowledge Source (a specialist agent in the Blackboard pattern)
JSON	JavaScript Object Notation; the format used for tool calls
API	Application Programming Interface; an external service an agent can call
RAG	Retrieval-Augmented Generation; using a search tool to feed facts to an LLM